

مینی پروژه

سیستم ذخیره پایگاه داده

عنوان درس : پیاده سازی سیستم های پایگاه داده

نام استاد : زهرا فامیل ستاری

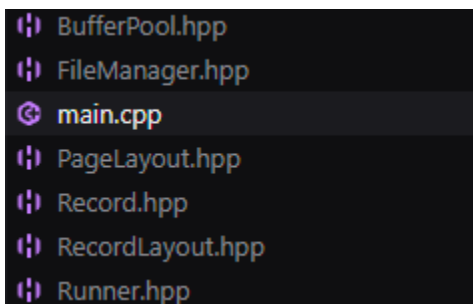
پیاده سازی و اجرا : مهرداد ورمزیار

بهار ۱۴۰۵

مقدمه :

در این پروژه، ما یک سیستم مدیریت پایگاه داده ساده را پیاده‌سازی کردیم که قابلیت ذخیره‌سازی و بازیابی رکوردها را در فایل‌های باینری دارد. این سیستم از صفحات با اندازه ثابت ۵۱۲ بایت استفاده می‌کند و هر صفحه می‌تواند حداکثر ۱۶ رکورد را در خود ذخیره کند. ما ماژول‌های مختلفی از جمله سریالایزر برای تبدیل رکوردها به آرایه‌های بایت، مدیریت صفحات برای افزودن و بازیابی رکوردها در صفحات، و مدیریت بافر با استفاده از الگوریتم LRU را پیاده‌سازی کردیم. همچنین توابعی برای نوشتن و خواندن صفحات از فایل دیسک ایجاد کردیم. در نهایت، با وارد کردن ۲۰ رکورد دانشجویی در فایل students.db، عملکرد سیستم را آزمایش کرده و توانستیم رکورد مورد نظر را پیدا کرده و نمایش دهیم. این پروژه مفاهیم بنیادین پایگاه داده مانند مدیریت حافظه کش، سریالایزیشن، و مدیریت صفحات را به خوبی نشان می‌دهد.

ماژول های طراحی شده :



در ادامه به بررسی و توضیح جامعی از ۴ بخش اصلی می پردازیم و در نهایت دو رکورد runner , header را بررسی می کنیم.

بررسی BufferPool :

کلاس BufferPool یکی از حیاتی‌ترین بخش‌های پروژه ما است که مسئولیت مدیریت حافظه کش (Cache) را بر عهده دارد. در یک پایگاه داده واقعی، خواندن و نوشتن مستقیم از روی دیسک برای هر درخواست بسیار کند است، بنابراین ما یک لایه میانی به نام بافر پول ایجاد کردیم تا صفحات پرکاربرد را در حافظه اصلی نگهداری کنیم. این کلاس دارای چهار عضو اصلی است: ظرفیت حداکثر تعداد صفحات قابل ذخیره در کش، یک دیکشنری به نام pages که شماره صفحه را به محتوای بایستی آن نگاشت می‌کند، یک لیست به نام lru که ترتیب دسترسی به صفحات را از قدیمی‌ترین تا جدیدترین حفظ می‌کند، و یک set به نام dirty که صفحات اصلاح شده (داغ) را نگهداری می‌کند. الگوریتم جایگزینی ما LRU (کمترین استفاده اخیر) است به این معنی که هر زمان کش پر شود، صفحه‌ای که طولانی‌ترین زمان از دسترسی به آن گذشته بود از حافظه خارج می‌شود.

متد getPage مهمترین تابع این کلاس است. وقتی صفحه‌ای درخواست می‌شود، ابتدا بررسی می‌شود که آیا از قبل در کش موجود است یا خیر. اگر موجود باشد، آن صفحه به انتهای لیست LRU منتقل می‌شود (نشانه استفاده اخیر) و بدون نیاز به خواندن از دیسک برگردانده می‌شود. اگر صفحه در کش نباشد، باید از دیسک خوانده شود و اگر جای کافی در کش نباشد، قدیمی‌ترین صفحه از لیست LRU حذف می‌شود. قبل از حذف، اگر آن صفحه در set (dirty) وجود داشته باشد، ابتدا محتویات آن به دیسک نوشته می‌شود تا اطلاعات از دست نرود، سپس صفحه از حافظه پاک می‌شود. متد markDirty برای علامت‌گذاری صفحاتی که تغییر کرده‌اند استفاده می‌شود تا هنگام حذف یا بستن برنامه، حتماً روی دیسک ذخیره شوند. متد flushAll نیز تمام صفحات داغ را یکجا روی دیسک می‌نویسد و سپس set داغ را خالی می‌کند. این طراحی باعث می‌شود که عملیات خواندن و نوشتن پایگاه داده به طور چشمگیری سریع‌تر شود و در عین حال سازگاری داده‌ها نیز حفظ گردد. در پروژه ما ظرفیت بافر پول برابر ۳ در نظر گرفته شده است و با وارد کردن ۲۰ رکورد، عملکرد LRU و فرایند خارج کردن صفحات از حافظه به خوبی قابل مشاهده بود.

برسی FileManager :

کلاس FileManager مسئول انجام عملیات سطح پایین ورودی و خروجی روی فایل‌های دیسک است. در پروژه ما، داده‌ها در فایلی به نام students.db ذخیره می‌شوند و این کلاس دو وظیفه اصلی دارد: نوشتن صفحات روی دیسک و خواندن صفحات از دیسک. از آنجایی که در پایگاه داده هر صفحه اندازه ثابتی دارد (در این پروژه ۵۱۲ بایت)، ما می‌توانیم با استفاده از ضریب ثابت، مکان دقیق هر صفحه را در فایل محاسبه کنیم.

متد writePage مسئول نوشتن یک صفحه در فایل است. این متد پارامترهای زیر را دریافت می‌کند: نام فایل، شماره صفحه و محتوای بایتی صفحه که یک بردار از std::byte به اندازه ۵۱۲ بایت است. ابتدا فایل را در حالت binary و با قابلیت خواندن و نوشتن باز می‌کند و اگر فایل وجود نداشته باشد، آن را می‌سازد. سپس با استفاده از تابع seekp، به موقعیت دقیق صفحه در فایل میرود که از فرمول offset برابر با page_num ضربدر ۵۱۲ محاسبه می‌شود. بعد از آن، با استفاده از تابع write، دقیقاً ۵۱۲ بایت از حافظه را روی دیسک می‌نویسد و در نهایت فایل را می‌بندد. این روش باعث می‌شود که ما بتوانیم هر صفحه را مستقل از صفحات دیگر به‌روزرسانی کنیم بدون اینکه نیاز به بازنویسی کل فایل باشد.

متد readPage نیز عملیات معکوس را انجام می‌دهد، یعنی یک صفحه مشخص را از روی دیسک خوانده و به حافظه اصلی می‌آورد. این متد فایل را در حالت خواندنی باز می‌کند، سپس با استفاده از تابع seekg به موقعیت دقیق صفحه در فایل می‌رود و با تابع read، دقیقاً ۵۱۲ بایت از فایل را خوانده و به صورت یک بردار از std::byte برمی‌گرداند. یکی از نکات مهم در این متد این است که اگر فایل وجود نداشته باشد یا صفحه درخواستی از انتهای فایل بزرگتر باشد، متد باید به درستی خطا را مدیریت کند. در پیاده‌سازی ما، فرض بر این است که صفحات به ترتیب و پشت سر هم در فایل قرار گرفته‌اند و دسترسی به هر صفحه با زمان تقریباً ثابت $O(1)$ انجام می‌شود.

مزیت اصلی استفاده از FileManager این است که جزئیات کار با فایل و محاسبه offset صفحات را از سایر بخش‌های سیستم مخفی می‌کند و یک رابط ساده و تمیز برای لایه‌های بالاتر مانند BufferPool فراهم می‌آورد. همچنین با استفاده از این طراحی، اگر در آینده بخواهیم فرمت ذخیره‌سازی یا روش آدرس‌دهی صفحات را تغییر دهیم، تنها کافی است این کلاس را اصلاح کنیم بدون اینکه سایر بخش‌های سیستم دست بخورند. در پروژه ما، کلاس BufferPool از طریق یک شی از FileManager برای خواندن صفحاتی که در کش موجود نیستند و همچنین نوشتن صفحات داغی که از حافظه خارج می‌شوند، استفاده می‌کند. این لایه‌بندی مناسب یکی از اصول مهم مهندسی نرم‌افزار یعنی جداسازی دغدغه‌ها را به خوبی رعایت کرده است.

کلاس PageLayout مسئول سازماندهی و مدیریت رکوردها درون یک صفحه با اندازه ثابت ۵۱۲ بایت است. در طراحی ما، هر صفحه از دو بخش اصلی تشکیل شده است: هدر ۴ بایتی که تعداد رکوردهای موجود در صفحه را ذخیره می‌کند، و بخش داده که حاوی رکوردهای واقعی است. با توجه به اینکه هر رکورد سریالایز شده دقیقاً ۳۰ بایت می‌کند، حداکثر ظرفیت هر صفحه ۱۶ رکورد خواهد بود زیرا ۴ بایت هدر به اضافه ۱۶ ضربدر ۳۰ بایت برابر با ۴۸۴ بایت می‌شود که در محدوده ۵۱۲ بایت قرار می‌گیرد و حدود ۲۸ بایت فضای خالی باقی می‌ماند (که برای توسعه‌های آینده می‌توان از آن استفاده کرد).

متد `init_page` یک صفحه جدید و خالی ایجاد می‌کند. این متد یک بردار ۵۱۲ بایتی از `std::byte` تولید می‌کند که تمام بایت‌ها با مقدار صفر پر شده‌اند. سپس ۴ بایت اول را که نقش هدر صفحه را دارند، با مقدار صفر مقداردهی می‌کند تا نشان دهد که تعداد رکوردهای موجود در این صفحه صفر است. این صفحه آماده است تا رکوردهای جدید به آن اضافه شوند.

متد `addRecordToPage` مهمترین تابع این کلاس است که وظیفه افزودن یک رکورد جدید به صفحه را بر عهده دارد. این متد ابتدا تعداد رکوردهای فعلی صفحه را از ۴ بایت اول می‌خواند. اگر تعداد رکوردها به ۱۶ رسیده باشد، یعنی صفحه پر است و متد یک خطای `overflow` پرتاب می‌کند. در غیر این صورت، محل درج رکورد جدید با فرمول `offset` برابر با ۴ به اضافه تعداد رکوردهای فعلی ضربدر ۳۰ محاسبه می‌شود. سپس بایت‌های رکورد (که دقیقاً ۳۰ بایت هستند) در این مکان کپی می‌شوند. در نهایت، شمارنده رکوردها یک واحد افزایش داده می‌شود و مقدار جدید در هدر صفحه (۴ بایت اول) به صورت `little-endian` نوشته می‌شود. صفحه به‌روز شده به عنوان خروجی بازگردانده می‌شود.

متد `getRecordFromPage` وظیفه بازیابی یک رکورد از صفحه را بر عهده دارد. این متد شماره اسلات (از ۱ تا ۱۶) را دریافت کرده و با استفاده از فرمول `offset` برابر با ۴ به اضافه (شماره اسلات منهای یک) ضربدر ۳۰، موقعیت شروع رکورد را محاسبه می‌کند. سپس ۳۰ بایت متوالی را از صفحه خوانده و به صورت یک بردار از `std::byte` برمی‌گرداند. نکته مهم این است که این متد هیچ بررسی‌ای برای بودن اسلات انجام نمی‌دهد و فرض می‌کند که اسلات درخواستی حاوی داده معتبر است.

مزیت اصلی این طراحی، دسترسی مستقیم و با زمان ثابت $O(1)$ به هر رکورد درون صفحه است. برخلاف روش‌های متغیر طول که باید لیست پیوندی از رکوردها را پیمایش کرد، در این روش با یک محاسبه ساده می‌توان محل دقیق هر رکورد را پیدا کرد. همچنین از آنجایی که اندازه همه رکوردها یکسان است، مدیریت فضای خالی و اضافه کردن رکوردهای جدید بسیار ساده می‌شود. این طراحی به اصطلاح `slot-based page layout` نام دارد و در بسیاری از سیستم‌های پایگاه داده واقعی مانند SQLite نیز استفاده می‌شود. هرچند که این روش مقداری فضای ذخیره‌سازی را به دلیل `padding` و اندازه ثابت فیلدها هدر می‌دهد، اما سادگی و سرعت بالای آن این عیب را جبران می‌کند. در پروژه ما، با استفاده از این کلاس توانستیم ۲۰ رکورد را به راحتی در صفحات مختلف ذخیره کرده و بدون هیچ گونه عملیات جستجوی پیچیده، رکورد شماره ۵ را بازیابی کنیم.

کلاس RecordLayout مسئول تبدیل رکوردهای ساختاریافته به آرایه‌های بایت (سریالایزیشن) و برعکس (دیسریالایزیشن) است. در پایگاه داده، داده‌ها باید به صورت آرایه‌ای از بایت‌ها روی دیسک ذخیره شوند و این کلاس دقیقاً همین کار را انجام می‌دهد. رکورد ما شامل چهار فیلد است: شناسه (id) که یک عدد صحیح ۴ بایتی است، نام (name) که یک رشته است، نمره (grade) که یک عدد اعشاری ۴ بایتی است، و وضعیت فعال (isActive) که یک مقدار بولین ۱ بایتی است. همچنین در انتهای هر رکورد یک بایت به نام null_bitmap قرار دارد که همیشه صفر است و برای توسعه‌های آینده در نظر گرفته شده است. بنابراین هر رکورد در مجموع ۳۰ بایت می‌کند (۴ + ۲۰ + ۱ + ۱). متد serializeRecord یک رکورد از نوع record را دریافت کرده و یک بردار از std::byte با اندازه ۳۰ بایت برمی‌گرداند. در مرحله اول، شناسه ۴ بایتی به صورت little-endian در خروجی قرار می‌گیرد؛ یعنی کم‌ارزش‌ترین بایت در اندیس صفر و پرارزش‌ترین بایت در اندیس سه ذخیره می‌شود. در مرحله دوم، فیلد name که یک رشته است، باید دقیقاً ۲۰ بایت شود. اگر طول نام کمتر از ۲۰ باشد، انتهای آن با کاراکتر فاصله (space) پر می‌شود و اگر بیشتر از ۲۰ باشد، برش داده می‌شود. سپس کاراکترها یک به یکی به خروجی اضافه می‌شوند. در مرحله سوم، نمره اعشاری ۴ بایتی با استفاده از تابع memcpy به آرایه بایت تبدیل می‌شود و به خروجی اضافه می‌شود. در مرحله چهارم، وضعیت فعال به صورت یک بایت (۱ برای true و ۰ برای false) ذخیره می‌شود. در نهایت، یک بایت صفر به عنوان null_bitmap به انتهای رکورد اضافه می‌شود. استفاده از little-endian در ذخیره‌سازی اعداد صحیح و اعشاری باعث می‌شود که فایل‌های تولید شده در سیستم‌های مختلف (مانند ویندوز و لینوکس) که هر دو معمولاً little-endian هستند (قابل خواندن باشند). متد deserializeRecord عملیات معکوس را انجام می‌دهد. این متد یک بردار از std::byte به اندازه ۳۰ بایت را دریافت کرده و رکورد اصلی را بازسازی می‌کند. ابتدا ۴ بایت اول را به صورت little-endian خوانده و به یک عدد صحیح تبدیل می‌کند. سپس ۲۰ بایت بعدی را به صورت کاراکتر خوانده و به یک رشته تبدیل می‌کند و در نهایت فاصله‌های انتهایی را حذف می‌کند تا نام اصلی بازگردانده شود. بعد از آن، ۴ بایت بعدی را با استفاده از memcpy به یک عدد اعشاری تبدیل می‌کند. سپس بایت بعدی را خوانده و به مقدار بولین تبدیل می‌کند. بایت آخر که null_bitmap است، خوانده می‌شود اما استفاده‌ای در برنامه فعلی ندارد. مزیت اصلی این طراحی، مستقل بودن فرمت ذخیره‌سازی از زبان برنامه‌نویسی است. فایل باینری تولید شده توسط برنامه سی‌پلاس‌پلاس ما را می‌توان با زبان‌های دیگر مانند پایتون یا جاوا نیز خواند، به شرطی که همان قوانین little-endian رعایت شود. همچنین استفاده از اندازه ثابت برای فیلد name باعث می‌شود که هر رکورد دقیقاً اندازه مشخصی داشته باشد و بتوان با محاسبه افسست، مستقیماً به رکوردهای مختلف دسترسی پیدا کرد بدون نیاز به پیمایش. این روش سریالایزیشن ساده اما کارآمد، یکی از روش‌های رایج در سیستم‌های پایگاه داده تعبیه‌شده مانند SQLite است و توازن خوبی بین سادگی پیاده‌سازی، سرعت اجرا و استفاده از فضای ذخیره‌سازی برقرار می‌کند. در پروژه ما، برای ذخیره ۲۰ رکورد دانشجویی و بازیابی رکورد شماره ۵، از همین کلاس استفاده شد و عملکرد صحیح آن به اثبات رسید.

برسی Record , Runner :

Record یک struct است که دارای آیدی و نام و نمره و فعال بودن یا نبودن یک دانشجو است که برای ذخیره سازی راحت تر و بهینه تر در کلاس RecordLayout استفاده می شود .

کلاس Runner صرفاً یک تست را اجرا می کند و دلیل استفاده رعایت پارادایم oop برای طراحی برنامه بود و پیچیدگی خاصی ندارد.

پایان